

Module 15 - chapter 1

Introduction to Ansible

Introduction to Ansible



- ✓ Real Life Use Cases
- ✓ Ansible Components
 - Modules & Playbook
- ✓ Ansible with Docker
- ✓ Ansible vs. alternative tools

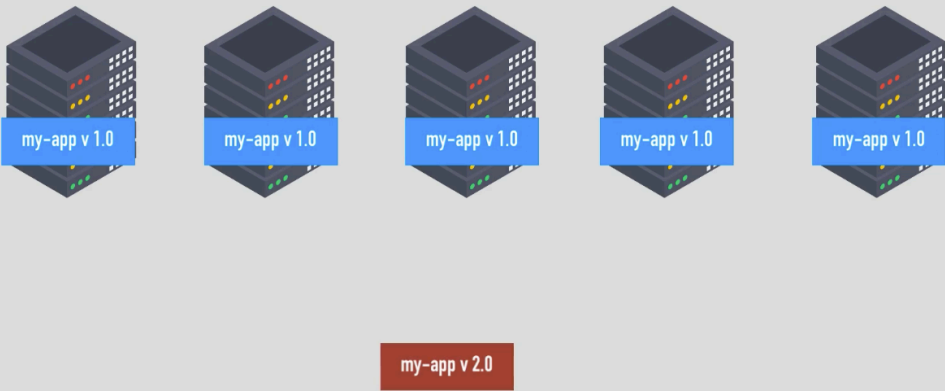
Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

Ansible is a tool that help us automate IT tasks!

What IT tasks?

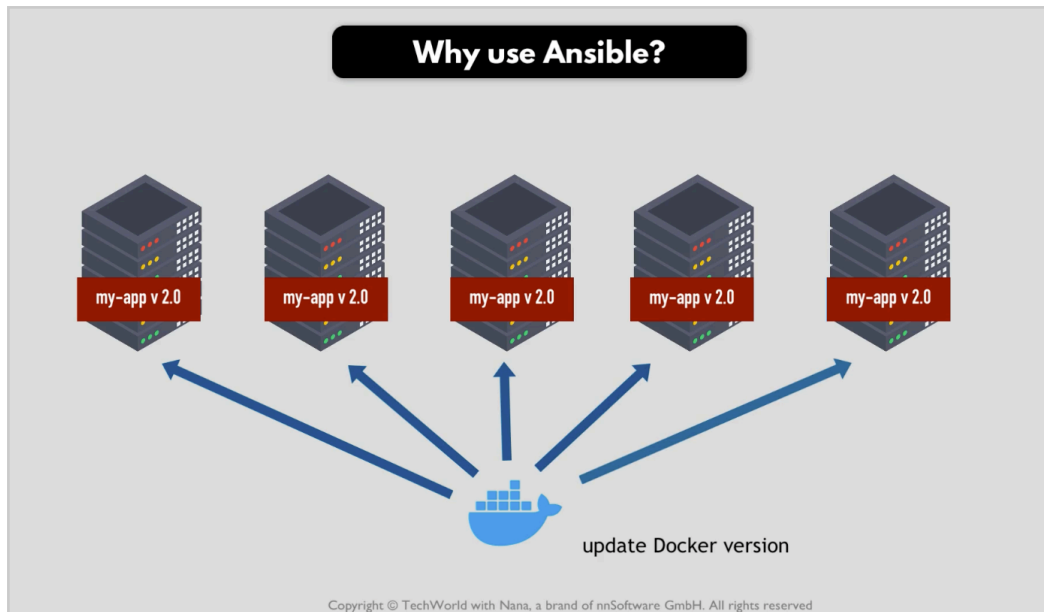
Let's imagine we have our application running in several servers and now we have a new version released of the application that needs to be deployed in the servers and replace the old app version

Why use Ansible?

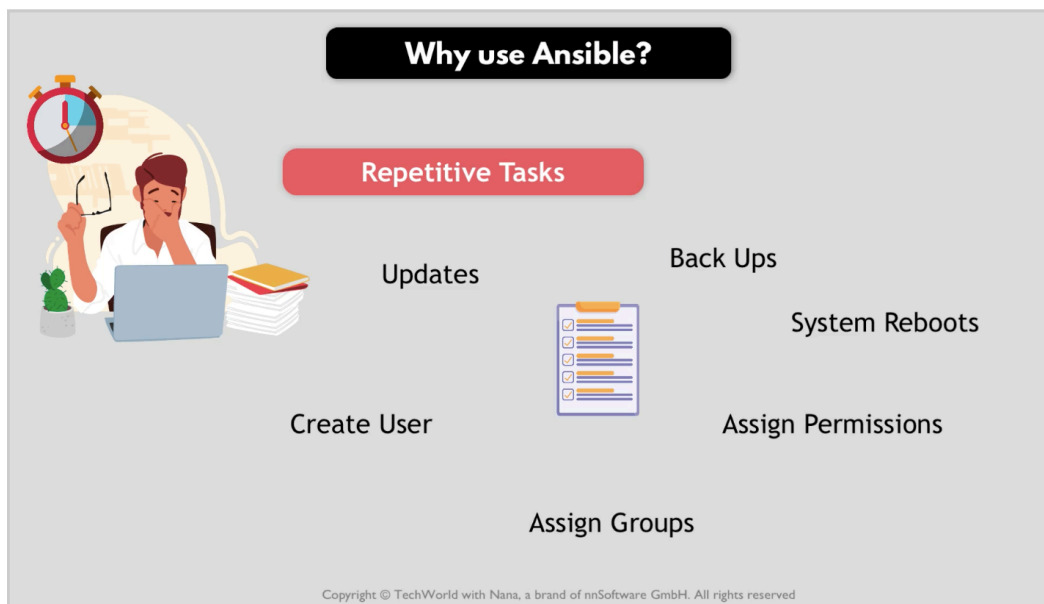


The diagram illustrates a deployment scenario. At the top, five server icons are shown, each labeled 'my-app v 1.0'. Below them, a single server icon is shown, labeled 'my-app v 2.0'. This represents the process of updating the application version across multiple servers.

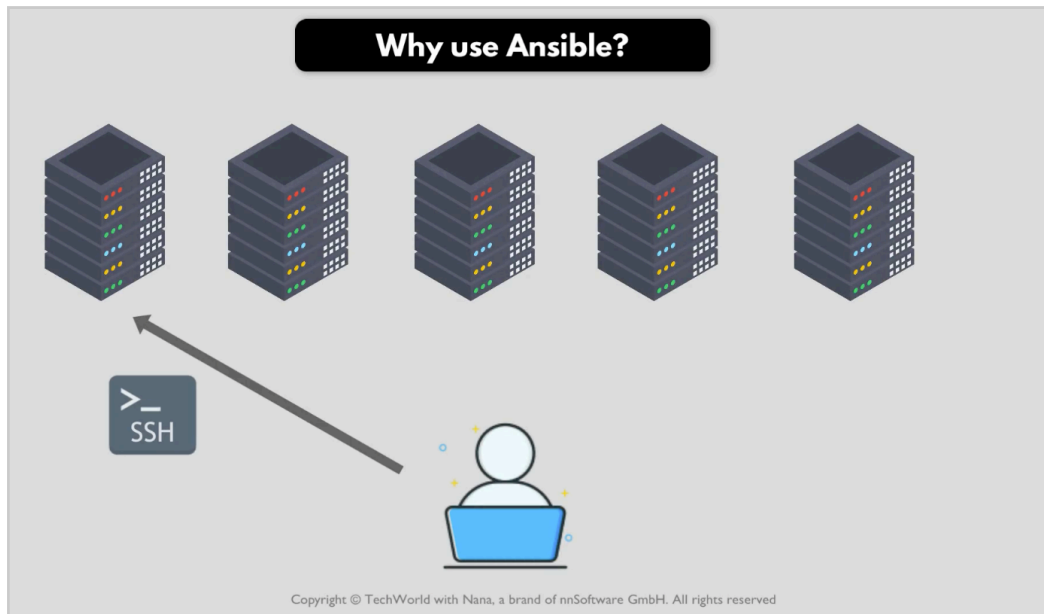
Or let's imagine we need to update the docker version in each of the servers



Other repetitive tasks that usually system administrators do manually



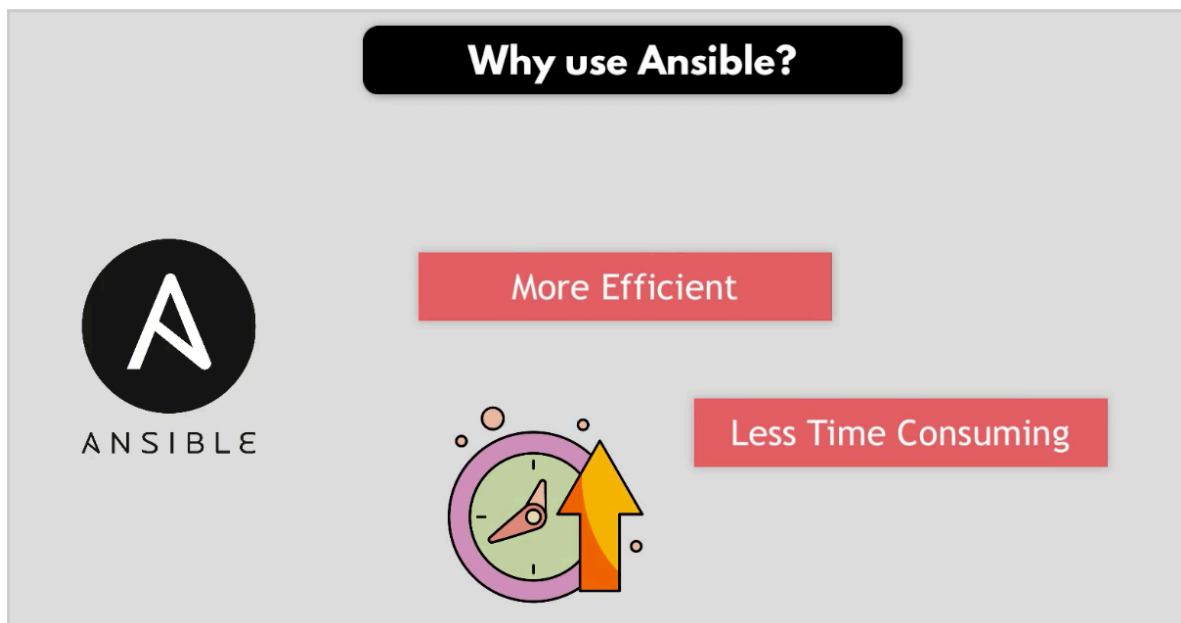
If done manually, each server needs to be ssh into and get the tasks done, one by one



And next time configuration needs an update for example then the whole process is repeated, ssh into one server -> configure then ssh into the next ssh -> configure etc.

And if the configuration involves different steps then we need to remember what we did last time, which are usually documented somewhere by the DevOps Eng.

Ansible makes life easier in 4 different ways!



1. Execute tasks from your own machine on all the servers remotely instead of stinging into each server.

1

Execute tasks from your own machine



Instead of SSHing into all remote servers

2. All the steps of writing the Configuration?installation/Deployment can be written in a single YAML file

2

Configuration/Installation/Deployment steps in a single
YAML File



Instead of manual and shell scripts

3. We can use the same file to execute the same task multiple times on the same server when we are doing re-upgrade for example or on different environments

3

Re-use same file multiple times and for different environments



4. More reliable and less likely prone to errors -> instead of human errors

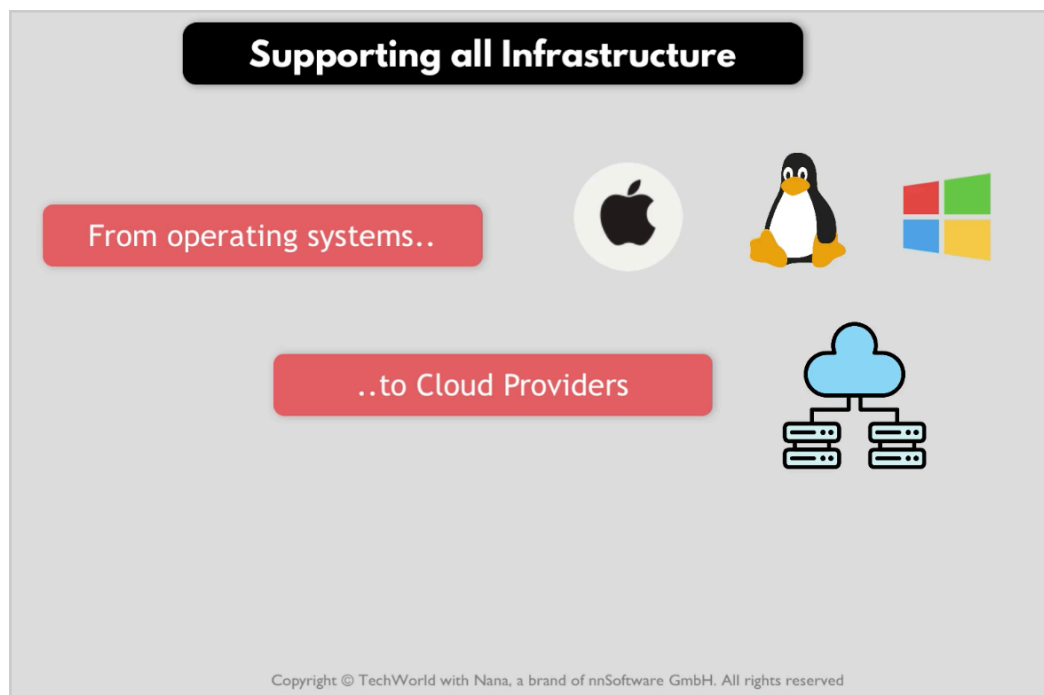
4

More reliable and less likely for errors

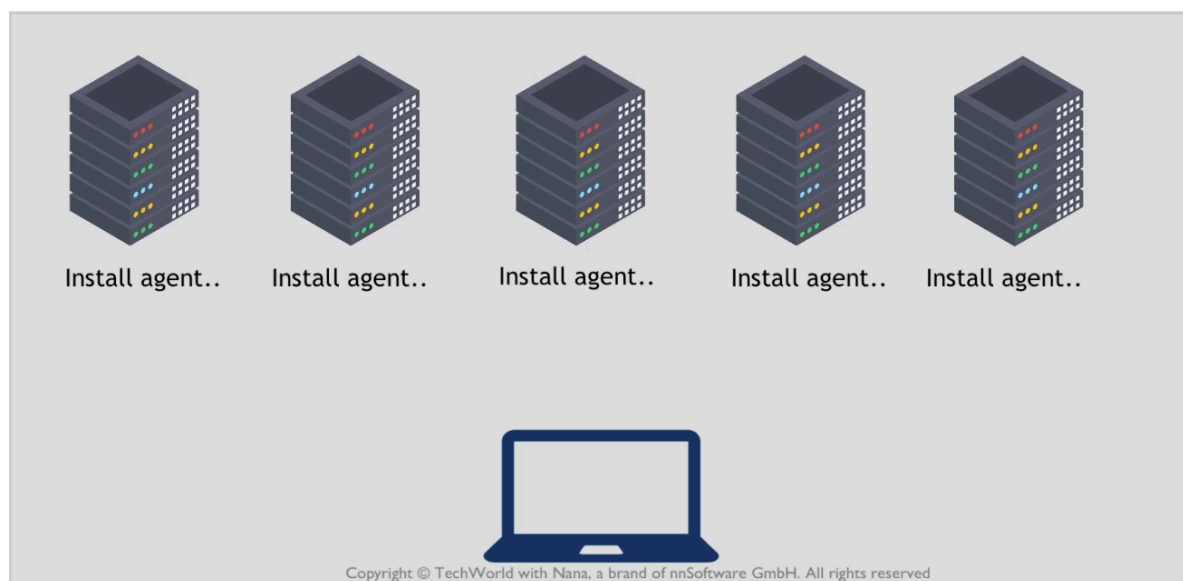


Supports all infra

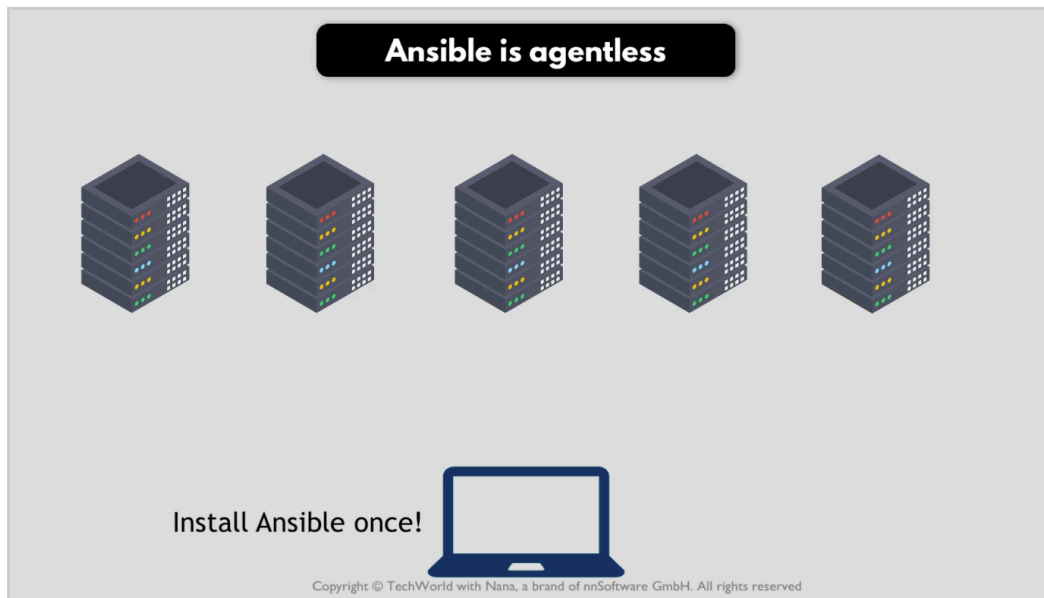
From OS to Cloud providers and bare metal.



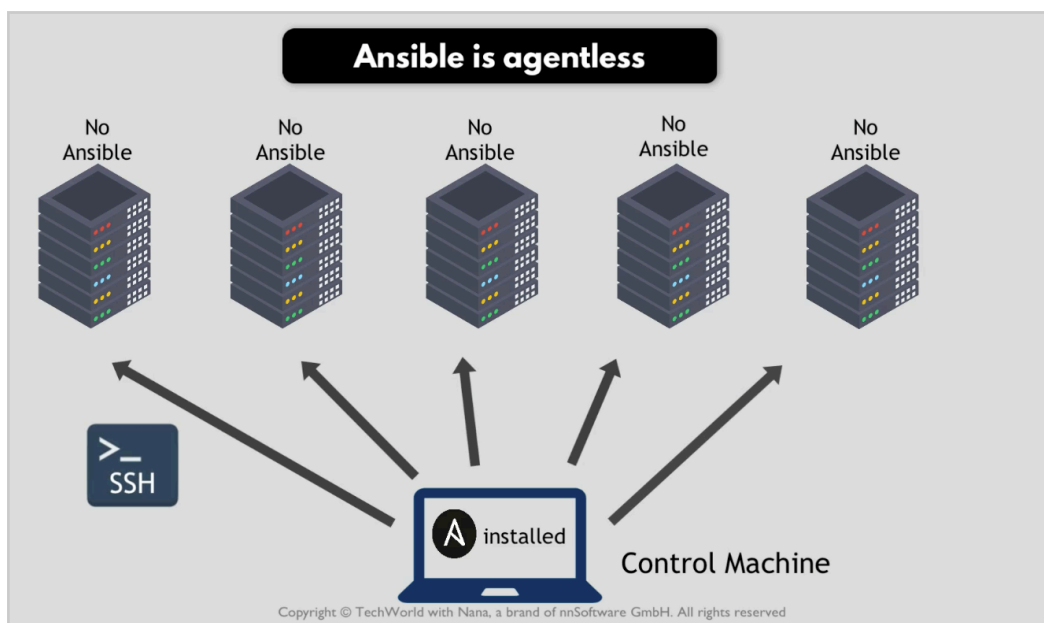
Usually when we want to use a tool on a machine we need to go to the machine and install an agent for the tool there -> it needs an agent.



Ansible is **AGENTLESS** we don't need to install an agent for the tool in the machine. We just install Ansible on One machine which becomes the 'control machine' and this could be our laptop



And the control machine will manage a whole fleet of target servers/machines remotely via ssh



So we don't have a deployment effort at the beginning and no upgrade effort when we need to upgrade Ansible

Ansible is agentless

No deployment effort in beginning

No upgrade effort



Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

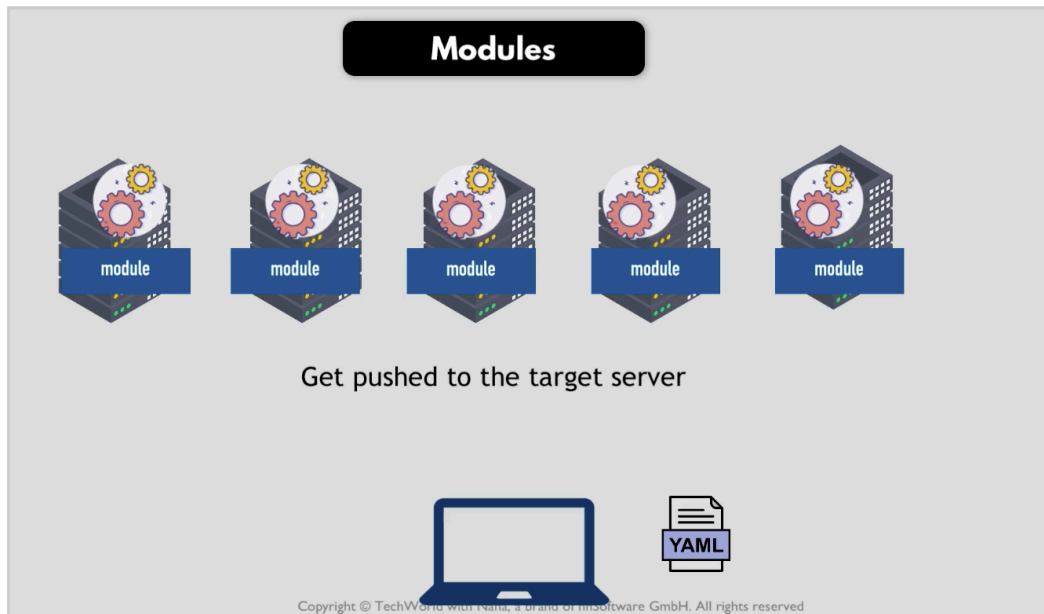
How Ansible works?

Ansible works with Modules -> small programs that do the actual work

Modules

= Small programs that do the actual work

The control machine pushes the module to each target server

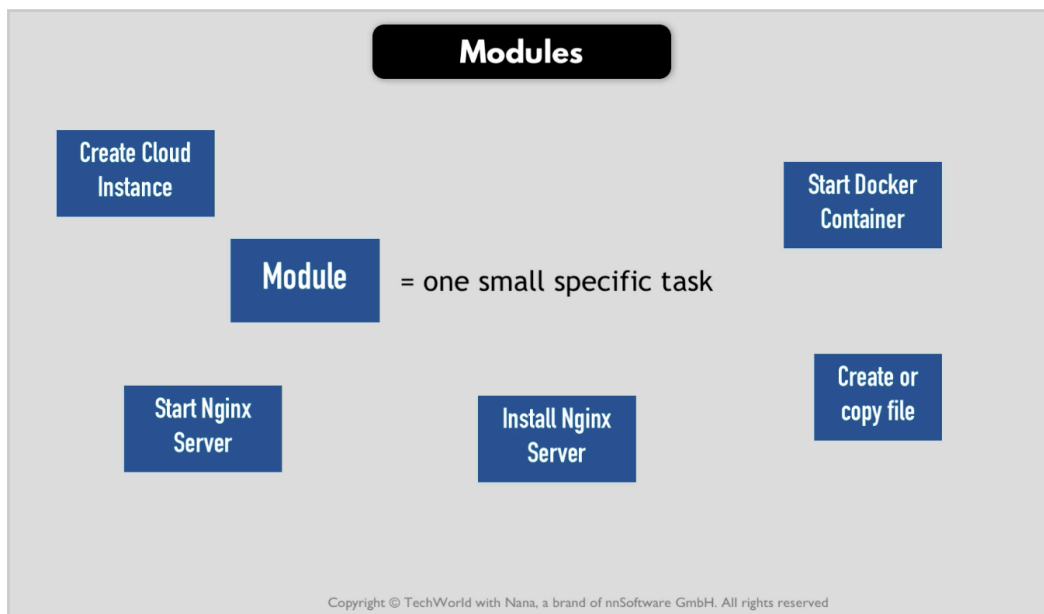


The modules do their work in the target server like:

- Install an app
- Stop a process
- Apply firewall rules

And when the module is done it gets removed.

Modules are granular -> one module does one small specific task

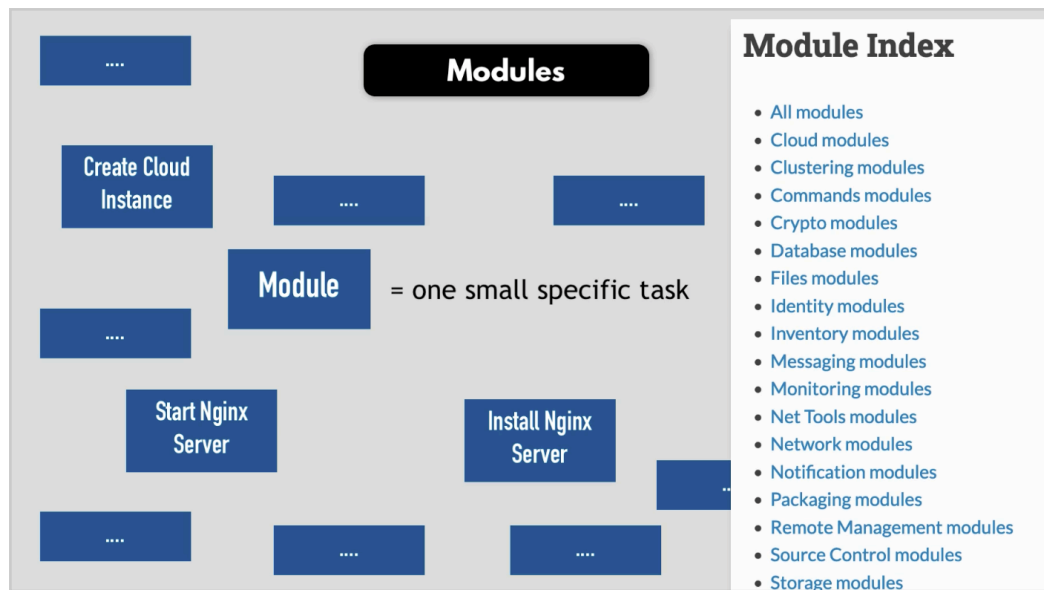


And Ansible has lots of modules, each executing specific tasks.

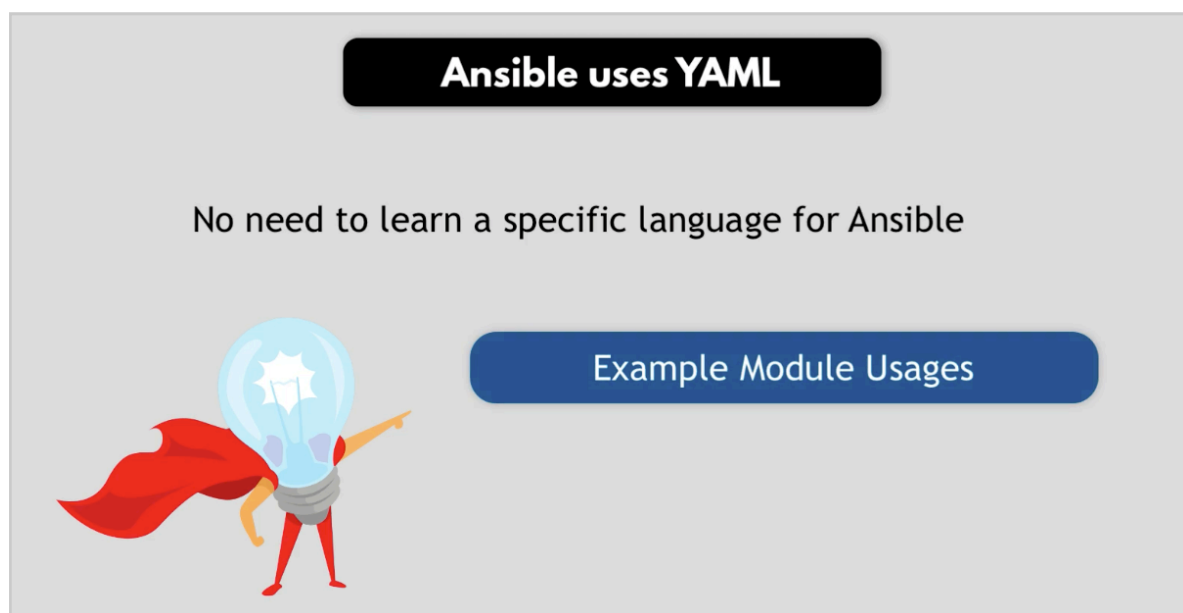
With Ansible we can execute any IT task because of all these modules.

It has modules for different cloud tools like AWS, etc, to work with many databases, to work with CI/CD tools and all of these can be found in Ansible

documentation



Ansible uses YAAML



Example Jenkins Module



Jenkins Module

jenkins_job

Creating a job

Deleting a job

```

# Create a jenkins job using basic authentication
- jenkins_job:
  config: "{{ lookup('file', 'templates/test.xml') }}"
  name: test
  password: admin
  url: http://localhost:8080
  user: admin

# Delete a jenkins job using the token
- jenkins_job:
  name: test
  token: asdfasfasdfasdfadfasdfasdfasdc
  state: absent
  url: http://localhost:8080
  user: admin

```

Example with Docker Module

Docker Module

docker_container

Create container:

Start container:

Apply configuration:

```

- name: Create a data container
  docker_container:
    name: mydata
    image: busybox
    volumes:
      - /data

- name: Start a container with a command
  docker_container:
    name: sleepy
    image: ubuntu:14.04
    command: ["sleep", "infinity"]

- name: Add container to networks
  docker_container:
    name: sleepy
    networks:
      - name: TestingNet
        ipv4_address: 172.1.1.18
        links:
          - sleeper
      - name: TestingNet2
        ipv4_address: 172.1.10.20

```

Example of a Postgres Module

Postgres Module

postgresql_table



Create table:

Set user privilege:

```
- name: Rename table foo to bar
postgresql_table:
  table: foo
  rename: bar

- name: Set owner to someuser
postgresql_table:
  name: foo
  owner: someuser

- name: Truncate table foo
postgresql_table:
  name: foo
  truncate: yes
```

So Modules are granular and specific -> how do we handle complex applications?

We group the modules! -> Ansible playbooks

Modules are granular and specific

module B

module A

module C

module D

- ▶ Multiple modules
- ▶ In a certain sequence
- ▶ Grouped together

Ansible Playbooks

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

The sequential modules are grouped in tasks

Ansible Playbooks

```
tasks:
- name: Rename table foo to bar
  postgresql_table:
    table: foo
    rename: bar

- name: Set owner to someuser
  postgresql_table:
    name: foo
    owner: someuser

- name: Truncate table foo
  postgresql_table:
    name: foo
    truncate: yes
```

1 Task = action to be performed

tasks

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

Each task has the module name

Ansible Playbooks

```
tasks:
- name: Rename table foo to bar
  postgresql_table:
    table: foo
    rename: bar

- name: Set owner to someuser
  postgresql_table:
    name: foo
    owner: someuser

- name: Truncate table foo
  postgresql_table:
    name: foo
    truncate: yes
```

Module name

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

And the task is executed with certain arguments

Ansible Playbooks

```
tasks:
  - name: Rename table foo to bar
    postgresql_table:
      {table: foo, rename: bar}

  - name: Set owner to someuser
    postgresql_table:
      {name: foo, owner: someuser}

  - name: Truncate table foo
    postgresql_table:
      {name: foo, truncate: yes}
```

Module name

Arguments

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

And also the task has a description

Ansible Playbooks

```
tasks:
  - name: Rename table foo to bar
    postgresql_table:
      {table: foo, rename: bar}

  - name: Set owner to someuser
    postgresql_table:
      {name: foo, owner: someuser}

  - name: Truncate table foo
    postgresql_table:
      {name: foo, truncate: yes}
```

Module name

Arguments

Description of task

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

The above is an example of 3 tasks all using Postgresql_table Module

- Rename table
- Set owner
- Truncate table

Ansible Playbooks

```
tasks:
  - name: Rename table foo to bar
    postgresql_table:
      table: foo
      rename: bar

  - name: Set owner to someuser
    postgresql_table:
      name: foo
      owner: someuser

  - name: Truncate table foo
    postgresql_table:
      name: foo
      truncate: yes
```

- Rename table
- Set Owner
- Truncate table

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

We can also can execute multiple modules in a sequence:

- Module file -> creates a directory
- Module yum -> installs
- Module service -> starts what we installed

And this is One configuration, each task is executed in order top to bottom

Ansible Playbooks

Execute multiple modules in a sequence:

```
tasks:
  - name: create directory for nginx
    file:
      { path: /path/to/nginx/dir
      { state: directory

  - name: install nginx latest version
    yum:
      { name: nginx
      { state: latest

  - name: start nginx
    service:
      { name: nginx
      { state: started
```

Module name

Arguments

1 configuration

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

Where should these tasks be executed? In which target machines?

We use the **HOSTS** attribute

And also we defined the USER the tasks will execute with the **remote_user** attribute

Ansible Playbooks

```
- hosts: databases
  remote_user: root

tasks:
  - name: Rename table foo to bar
    postgresql_table:
      table: foo
      rename: bar

  - name: Set owner to someuser
    postgresql_table:
      name: foo
      owner: someuser

  - name: Truncate table foo
    postgresql_table:
      name: foo
      truncate: yes
```

Where should these tasks be executed?

HOSTS

With which user should the tasks execute?

REMOTE_USER

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

In this example we use 'root' user but we can use a non-root user and then escalate to 'root' user privileges when need it.

Another important note is that YMAL is strict about indentation.

```
- name: Truncate table foo
  postgresql_table:
    name: foo
    truncate: yes
```

Strict indentation!

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

When we have repetition of values we can use variables ❤️

Ansible Playbooks

Use variables for repeating values

```
- hosts: databases
  remote_user: root

  tasks:
    - name: Rename table foo to bar
      postgresql_table:
        table: foo
        rename: bar

    - name: Set owner to someuser
      postgresql_table:
        name: foo
        owner: someuser

    - name: Truncate table foo
      postgresql_table:
        name: foo
        truncate: yes
```

```
- hosts: databases
  remote_user: root
  vars:
    tablename: foo
    tableowner: someuser

  tasks:
    - name: Rename table {{ tablename }} to bar
      postgresql_table:
        table: {{ tablename }}
        rename: bar

    - name: Set owner to someuser
      postgresql_table:
        name: {{ tablename }}
        owner: someuser

    - name: Truncate table
      postgresql_table:
        name: {{ tablename }}
        truncate: yes
```

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

PLAY -> which tasks + which hosts + which user

Ansible Playbooks

"Play"

- which tasks
- which hosts
- which user

```
- hosts: databases
  remote_user: root
  vars:
    tablename: foo
    tableowner: someuser

  tasks:
    - name: Rename table {{ tablename }} to bar
      postgresql_table:
        table: {{ tablename }}
        rename: bar

    - name: Set owner to someuser
      postgresql_table:
        name: {{ tablename }}
        owner: someuser

    - name: Truncate table
      postgresql_table:
        name: {{ tablename }}
        truncate: yes
```

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

PLAYBOOK

And in a single YAML file I can have One or multiple PLAY blocks -> PLAYBOOK

For example below there are two PLAY blocks, one for Webservers and one for Databases.

```

hosts: webservers
remote_user: root

tasks:
  - name: create directory for nginx
    file:
      path: /path/to/nginx/dir
      state: directory

  - name: install nginx latest version
    yum:
      name: nginx
      state: latest

  - name: start nginx
    service:
      name: nginx
      state: started

- hosts: databases
  remote_user: root

  tasks:
    - name: Rename table foo to bar
      postgresql_table:
        table: foo
        rename: bar

    - name: Set owner to someuser
      postgresql_table:
        name: foo
        owner: someuser

    - name: Truncate table foo
      postgresql_table:
        name: foo
        truncate: yes

```

Playbook = 1 or more Plays

Multiple Plays in a single YAML file

Play for Webservers

Play for Databases

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

A Playbook describes:

- How, in which order, where (which machines) and what modules should be executed.

The playbook orchestrates the module execution.

```

hosts: webservers
remote_user: root

tasks:
  - name: create directory for nginx
    file:
      path: /path/to/nginx/dir
      state: directory

  - name: install nginx latest version
    yum:
      name: nginx
      state: latest

  - name: start nginx
    service:
      name: nginx
      state: started

- hosts: databases
  remote_user: root

  tasks:
    - name: Rename table foo to bar
      postgresql_table:
        table: foo
        rename: bar

    - name: Set owner to someuser
      postgresql_table:
        name: foo
        owner: someuser

    - name: Truncate table foo
      postgresql_table:
        name: foo
        truncate: yes

```

Playbook = 1 or more Plays

Multiple Plays in a single YAML file

Play for Webservers

Play for Databases

Playbook describes:

- How and in which order
- At what time and where (on which machines)
- What (the modules) should be executed

Orchestrates the module execution ★

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

How to name the Plays in the Playbook?

We use the name attribute like we did for tasks.

The name summarises what the play does -> good practice

Good practice: Naming Plays

```
- name: install and start nginx server
hosts: webservers
remote_user: root

tasks:
  - name: create directory for nginx
    file:
      path: /path/to/nginx/dir
      state: directory

  - name: install nginx latest version
    yum:
      name: nginx
      state: latest

  - name: start nginx
    service:
      name: nginx
      state: started
```

```
- name: rename table, set owner and truncate it
hosts: databases
remote_user: root
vars:
  tablename: foo
  tableowner: someuser

tasks:
  - name: Rename table {{ tablename }} to bar
    postgresql_table:
      table: {{ tablename }}
      rename: bar

  - name: Set owner to someuser
    postgresql_table:
      name: {{ tablename }}
      owner: someuser

  - name: Truncate table
    postgresql_table:
      name: {{ tablename }}
      truncate: yes
```

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

Where do the values of the HOSTS attributes come from?

```
- name: install and start nginx
hosts: webservers
remote_user: root
```

Where do the hosts values come from? 🤔

```
- name: rename table, set owner and truncate it
hosts: databases
remote_user: root
vars:
```

The value is found in Ansible inventory list

Ansible has a host file that keeps the inventory -> all the machines that are involved in Ansible task executions

And we can define the entries in the host file as IP addresses or the hostnames

Ansible Inventory List

Hosts File

```
10.24.0.100

[webservers]
10.24.0.1
10.24.0.2

[databases]
10.24.0.7
10.24.0.8
```

```
10.24.0.100

[webservers]
web1.myserver.com
web2.myserver.com

[databases]
10.24.0.7
10.24.0.8
```

```
- hosts: databases
remote_user: root
```

```
hosts: webservers
remote_user: root
```

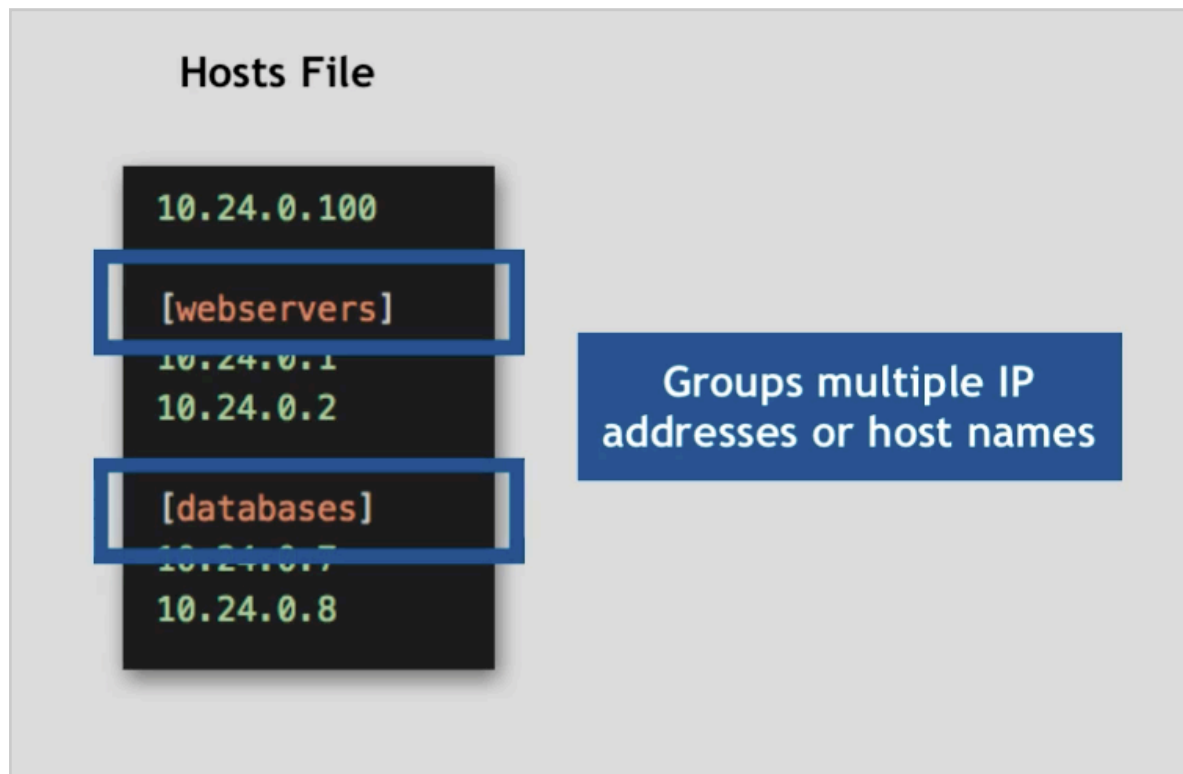
IP address or hostname

Inventory = all the machines involved in task executions

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

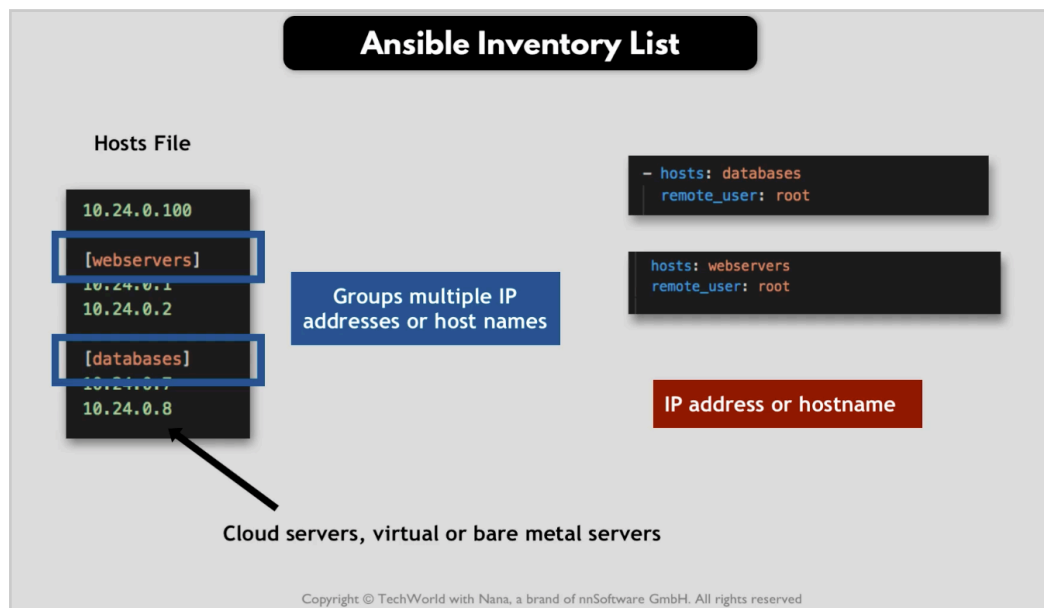
The syntax [**<hostname>**] groups multiple IP addresses or host names together

so we can reference them as a group



So If I need to execute a play for all the DBs then it is easier to call the `[databases]` group name than the list of IP addresses.

Those IP addresses or hostnames could be of cloud instances, virtual or bare metal servers.

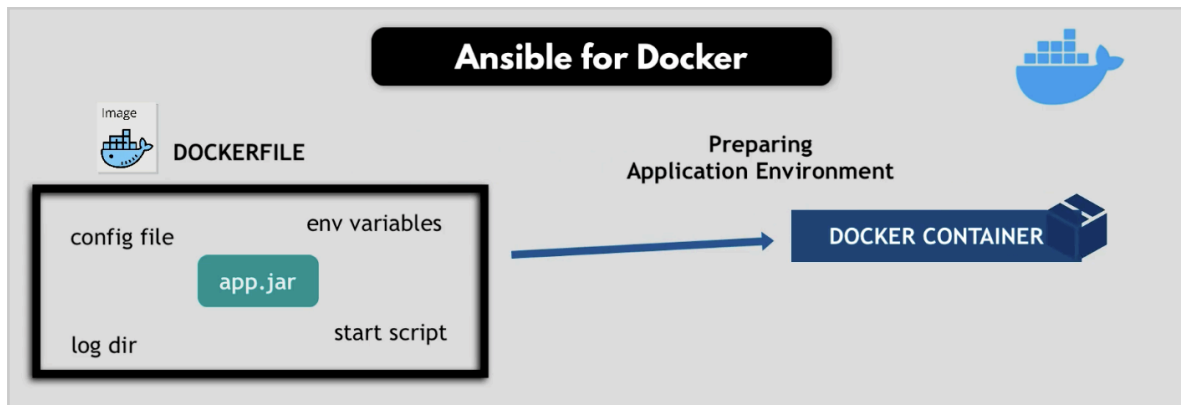


Ansible usage for Docker

When we want to create a docker container we use dockerfile which prepares

the application environment.

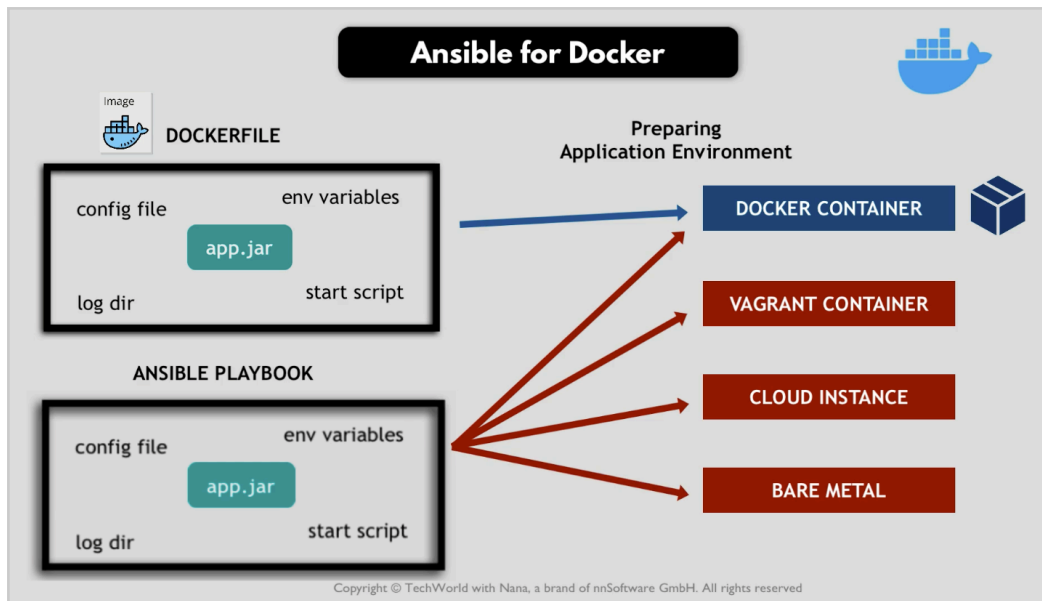
So we have the app artifact (like a .jar) and we defined the config file should be copied into the environment, maybe a directory should be created for the log files to be stored, also we define env vars and maybe we also copy a start script that should be executed before the app starts.



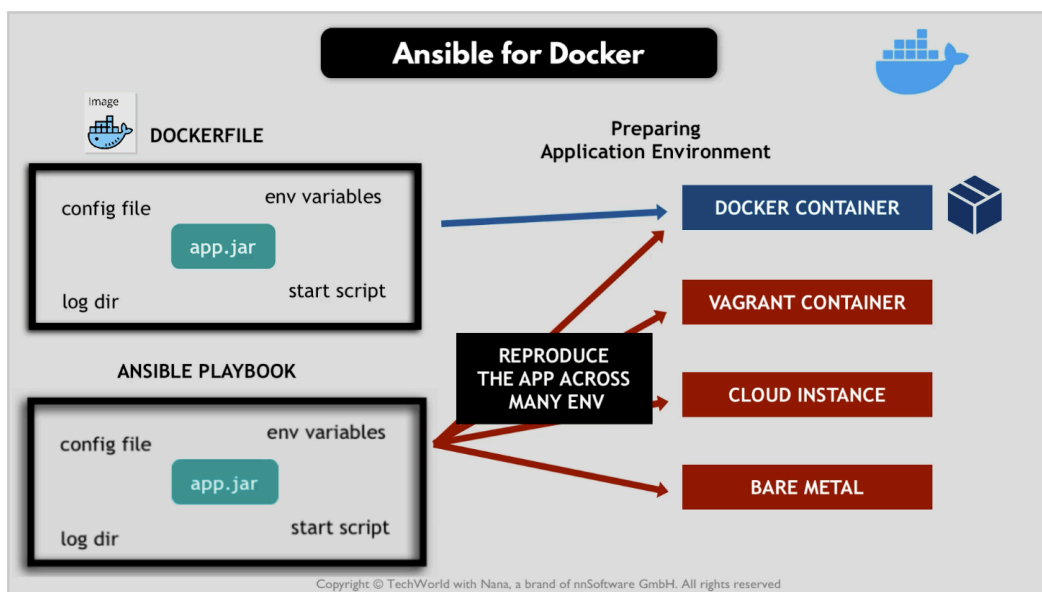
So we prepare the environment for the application startup. And with Dockerfile that will produce a docker container with the app and its prepared environment.

With ansible we can create an alternative to dockerfile that is more powerful -> we can take the same type of configuration with application artifact and instead of only creating a docker container we can create:

- Docker container
- Vagrant container
- Deploy the app on a cloud instance
- Or deploy the app on a bare metal server



Ansible allows us to reproduce the app across many environments not only docker container



— vagrant tangent —

Q:

I have this note: With ansible we can create an alternative to dockerfile that is more powerful -> we can take the same type of configuration with application artifact and instead of only creating a docker container we can create:

- Docker container
- Vagrant container
- Deploy the app on a cloud instance
- Or deploy the app on a bare metal server

can you explain, just want to be sure I am understanding as I don't know what a

vagrant conatiner is

A:

Quick clarification before the explanation: "Vagrant container" isn't quite the right term — Vagrant manages virtual machines (VMs), not containers. That distinction matters a lot for what your Ansible-based tool would actually be doing.

The core idea

What you're describing is real and used in practice: use Ansible as the "build" layer, then target different backends for the "run" layer. Instead of a Dockerfile (which only knows how to produce a Docker image), you write Ansible roles/playbooks that describe "install these packages, copy this artifact, configure this service" — and that same configuration can be applied to:

Target	What it actually is	How Ansible gets there
Docker container	A container image	Ansible can build images via <code>ansible-bakery</code> , or you provision a running container with the <code>docker</code> connection plugin
Vagrant "container"	Actually a VM (VirtualBox/libvirt/etc.)	Vagrant boots the VM, then hands it to Ansible as a provisioner (vagrant provision runs your playbook inside the VM)
Cloud instance	An EC2/GCE/Azure VM	Ansible provisions the cloud instance over SSH, same playbook
Bare metal	A physical server	Ansible provisions over SSH, same playbook

The insight is: Ansible playbooks are idempotent, declarative-ish configuration that works over SSH (or a container/API connection) regardless of what's underneath. A Dockerfile's RUN steps are locked to producing a Docker layer. An Ansible role isn't locked to anything — it just needs a connection to a target.

What "Vagrant container" really means

Vagrant's job is to spin up a disposable VM (via VirtualBox, libvirt, Hyper-V, etc.) from a base image ("box"), give it networking/shared folders, and then optionally run a provisioner against it — Ansible is one of the built-in provisioner options. So in your model:

- Vagrant = "give me a fresh VM to provision" (dev/test sandbox on your laptop)
- Docker = "give me a fresh container to provision" (lightweight, shares host kernel)
- EC2/cloud = "give me a fresh VM in the cloud to provision"
- Bare metal = "here's a real machine, provision it"

In all four cases, the thing doing the actual "install my app" work is the same Ansible playbook — only the provisioning target/connection method changes (Docker API, Vagrant/VirtualBox, cloud API + SSH, or plain SSH).

Why this is "more powerful" than a Dockerfile

- Dockerfile = single-target, single-abstraction (image layers only), limited logic (no real conditionals/loops/variables the way Ansible has).
- Ansible = full templating, variables, conditionals, idempotency checks, and — critically — connection plugins, so the same role runs against docker, local (Vagrant VM), or ssh (cloud/bare metal) with just an inventory change.

This is essentially the pattern behind tools like Packer (image building for multiple platforms) combined with Ansible provisioners, or how mature infra teams use one Ansible role tree for both "build a Docker image" and "configure a production VM."

Q:

maybe a need more detail or simpler words. so I have my dockerfile code that "has the app artifact (lie a .jar) and we defined the config file should be copied into the environment, maybe a directory should be created for the log files to be stored, also we define env vars and maybe we also copy a start script that should be executed before the app starts", if I set the same information in ansible I can create a docker file and also I can have vagrant VM that will run my app the way dockercontainer does?

A:

Yes, exactly — that's the right mental model. Let me map it line-by-line using your example.

Your Dockerfile, broken into steps

1. Copy the .jar artifact into the environment
2. Copy a config file
3. Create a directory for logs
4. Set environment variables
5. Copy a start script and run it before the app starts

Every one of these is just "do a thing to a machine" — it doesn't actually care if that machine is a Docker container, a VM, or a physical server. Docker just happens to express these as COPY, RUN mkdir, ENV, ENTRYPOINT.

Same steps, written as Ansible tasks

- name: Copy app jar
copy:
src: myapp.jar
dest: /opt/app/myapp.jar
- name: Copy config file
copy:
src: app-config.yml
dest: /opt/app/config/app-config.yml
- name: Create log directory
file:
path: /var/log/myapp
state: directory
- name: Set environment variables
copy:
content: "APP_ENV=production\n"
dest: /etc/environment
- name: Copy start script
copy:
src: start.sh
dest: /opt/app/start.sh
mode: '0755'
- name: Run start script
command: /opt/app/start.sh

This is now just a recipe, with no mention of Docker anywhere. It says "copy this, create that, set this env var, run this script" — same as before, just not locked into Dockerfile syntax.

Now the important part: where does this recipe run?

Ansible needs a target machine to SSH into (or connect to some other way). That target is the only thing that changes:

- Vagrant VM: Vagrant boots a VM (via VirtualBox), and Vagrant itself can hand that VM to Ansible to run the exact recipe above. Result: a full VM with your jar,

config, log folder, env vars, and running app — same end state as the container, just inside a VM instead of a container.

- Docker container: You'd run these same tasks against a running container (Ansible has a docker connection mode), or use them to assemble the image contents.

- Cloud instance (EC2 etc.): Ansible SSHes into the cloud VM once it's booted and runs the same recipe.

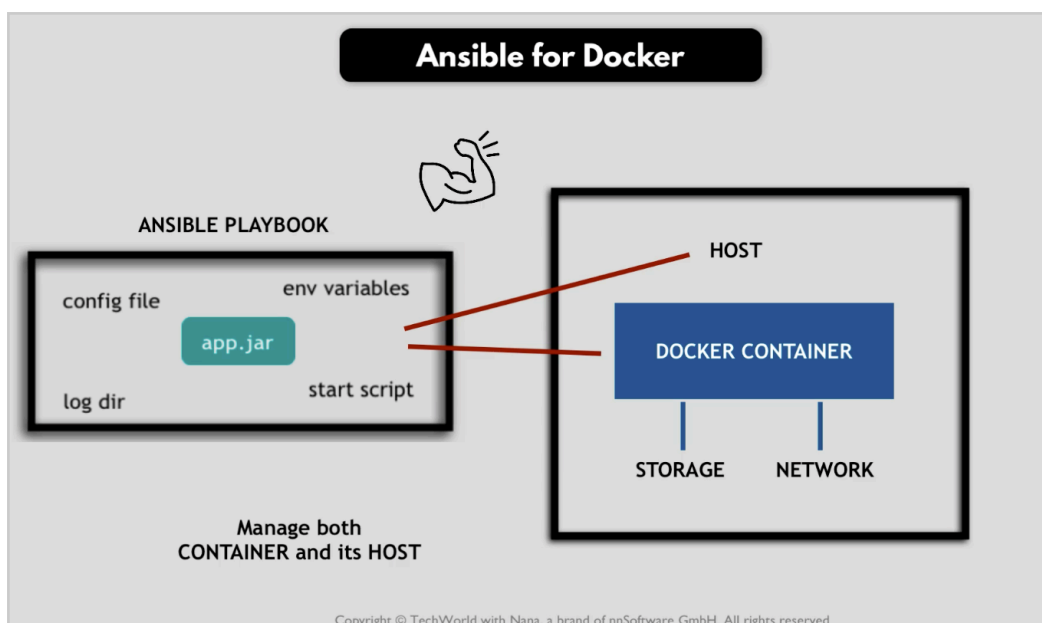
- Bare metal server: Same thing — SSH in, run the same recipe.

So yes: one Ansible playbook describing "artifact + config + logs dir + env vars + start script" can provision a Vagrant VM to run your app the same way the Docker container does — because both are just "a machine that ends up with those files, folders, variables, and a running process." The playbook doesn't know or care that one target is a container and the other is a VM; it just needs a connection to something with a filesystem and a shell.

The one thing to flag: a VM boots a full OS (its own kernel, systemd, etc.), while a container shares the host's kernel and is much lighter/faster to start. So functionally you get the same running app, but the Vagrant VM is heavier — that's the trade-off for the added flexibility.

— end target —

Another advantage of Ansible is that we not only manage the docker container but also the host of the docker container, so if the docker container has dependencies on the storage of the host or network then Ansible helps us to manage those parts as well



So in the PLAYBOOK we can manage both:

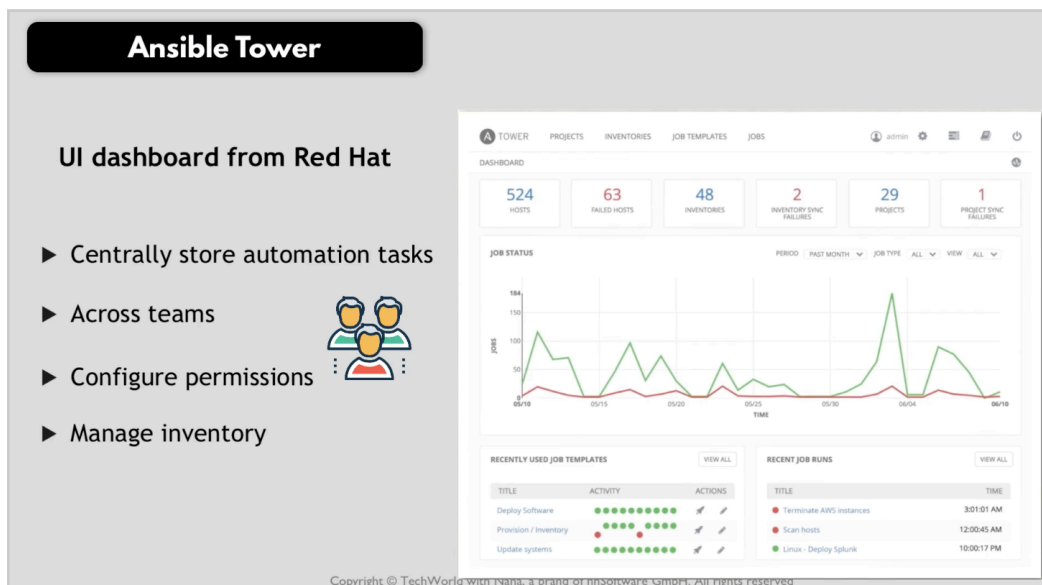
- Container
- Its host

Ansible Tower

UI dashboard from red hat.

We can centrally store all of our automation tasks across teams where we can:

- configure permissions for those teams
- Manage the inventory -> have an overview of which jobs have run and health status



Alternatives tools to Ansible

Puppet and Chef



- Puppet
- Chef

Comparable Tools

Ansible

- ▶ Simple YAML
- ▶ Agentless

Puppet and Chef

- ▶ Ruby - more difficult to learn
- ▶ Installation needed
- ▶ Requires managing updates on target servers